INGENIERÍA DE SOFTWARE
1ASI0730 - APLICACIONES WEB
PRÁCTICA CALIFICADA 1
202620

NRC: 8130
Profesor: Villafuerte Bazan, Oscar Ivan
Duración: 100 minutos
Indicaciones:

1. El examen consta de 1 pregunta, y tendrá 100 minutos para resolverlas.
2. La pregunta es de tipo Proyecto de Software y la entrega de su respuesta es a través de envío de archivo empaquetado **.zip** con proyecto de software en la Actividad PC1 (Práctica Calificada 1).

Enunciado:

Pregunta 1 (20 p.).

Caso Open Library API

Su cliente, Open Library API (https://openlibrary.org/developers/api), una plataforma de biblioteca virtual y pública con información de libros electrónicos desea promover el uso de su catálogo de libros clasicos.

Para ello, le solicita elaborar una aplicación web que obtenga la información de los libros accediendo al endpoint:

URL: https://openlibrary.org
Documentación:
https://openlibrary.org/developers/api
https://openlibrary.org/search.json?subject=classics&fields=key,title,author_name,cover_i,first_publish_year,language,number_of_pages_median
https://covers.openlibrary.org/b/id/{cover_i}-{tamaño}.jpg

Para el desarrollo web de lado web frontend, se ha seleccionado JavaScript como lenguaje de programación y Vue como Frontend Framework.

Se le encarga el desarrollo de una aplicación web que implemente las siguientes características:

Toolbar y Header:
- La aplicación debe presentar en la vista home un Toolbar en la parte superior. En la parte central, debe mostrar un logo (obtenido a través del servicio Logo.dev Logo API) y el título "Open Library Classic". En la parte izquierda, debe presentar un conjunto de *select buttons* (EN | ES) para la selección de idioma de la interfaz entre inglés o español.

Vista Principal (Book List):
- Dentro de la vista principal, la aplicación debe mostrar una sección con el título "Classic books" que contenga un conjunto de *cards* para presentar la información de cada libro registrado.
- Imagen: En la parte superior del card debe mostrarse la imagen del libro (cover_i).
- Encabezado: El título del libro (`title`) debe aparecer como título y la editora (`publisher`) como subtítulo.
- Contenido: La información adicional que ofrece la API (autor, fecha de publicación y números de páginas) debe incluirse en el contenido del card utilizando *human-friendly labels* traducidas según el idioma seleccionado. Se deben aplicar las mejores prácticas para cards y organización de información según *Material Design*.

Footer del Card:
- Izquierda: El elemento 'Language, el cual muestra todos los idiomas que tiene la obras language.
- Derecha: La opción 'Share Information' para compartir el libro utilizando la capacidad nativa de *share* del navegador web o copiar el enlace al portapapeles si la API nativa no está disponible.

Disposición y Comportamiento:
- Debe mostrar tres cards por fila.
- Al cambiar el idioma mediante los *select buttons*, la actualización debe reflejarse en todos los títulos, frases, etiquetas y textos fijos de la interfaz de usuario.

Footer General:
La parte inferior de la aplicación debe incluir un footer con dos líneas:
- Línea 1: Copyright © 2026 Open Library API. All rights reserved.
- Línea 2: Developed by seguido de sus datos personales (código de estudiante, nombres y apellidos).

Aspectos generales:
- El equipo requiere que rutas y claves no estén en hardcode en la aplicación, sino en variables de entorno, distinguiendo entornos de *development* y *production*.
- El equipo de IT de su cliente tomará en cuenta no solo el cumplimiento de las características funcionales, sino el diseño de interfaz de usuario, así como la estructura del proyecto, aplicación de convenciones de nomenclatura de objetos de programación en inglés, convenciones de nomenclatura de Vue.js, organización y eficiencia del código. Igualmente se tomará en cuenta la aplicación de patrones de diseño.

Restricciones técnicas:

Nombre su proyecto como *pc1**nrc**u**code*** donde ***nrc*** es el número de NRC y ***code*** es su código de estudiante en minúsculas (por ejemplo *pc18130u20161C993*).
Debe generar el proyecto con Vite y utilizar Vue 3.5, con Composition API. El equipo requiere que la interfaz de usuario esté basada en el tema **Material** utilizando la biblioteca de componentes de UI **PrimeVue**, mientras que para la comunicación con el backend debe apoyarse en **axios** (https://github.com/axios/axios). Debe incluir ARIA atributes en las vistas. La interfaz de usuario debe mostrar los textos en **inglés por defecto**. Utilice Vue-i18n (https://vue-i18n.intlify.dev/) para el soporte de internacionalización. La organización del proyecto debe ser domain-driven aplicando layered y component-based architecture, object-oriented programming y design patterns, considerando los sub-dominios ***shared*** (para elementos o componentes generales de la interfaz de usuario, como *layout*) y ***library*** (con los elementos correspondientes, como *book List*). Distribuya adecuadamente los elementos dentro de cada carpeta de sub-dominio, considerando carpetas como *presentation*, *infrastructure*, *aplication, domain*, así como subcarpetas según el tipo de layer según corresponda. Aplique buenas prácticas para nomenclatura lógica y física de clases y componentes. Aplique patrones de diseño, incluyendo *Request/Response*, *Resource* y *Assembler*. Aplique buenas prácticas y convenciones aplicadas en clase para nomenclatura lógica y física de clases y componentes. El proyecto de aplicación debe poder aperturarse sin problemas en JetBrains WebStorm.
Para la nomenclatura física de componentes, clases e interfaces utilice kebab-case. Aplique en los nombres físicos sufijos que indiquen el tipo de elemento: *.entity.js, .store.js, .assembler.js*. En el caso de entities, el sufijo *entity* no debe ser parte del nombre de la entidad cuando se declare la clase (Por ejemplo *book.entity* debería utilizarse en código de clase como *Person*). Para los componentes que elabore no se aplica sufijos .component o similar, solo se nombre con sustantivos simples o compuestos en kabab-case. Para la nomenclatura de componentes de PrimeVue dentro de los templates utilice nombres sustantivos en kebab-case. Asigne el prefijo pv- al momento de incorporar su uso en el proyecto (Por ejemplo, pv-card). En caso de que la información de los JSON objects que proporciona el API provider, incluidos en los response no cumplan con las convenciones de nomenclatura de objetos de JavaScript, ello no debe afectar la nomenclatura de los domain entities. Es responsabilidad de los Assembler objects la asignación de información a los atributos que correspondan entre resource objects y domain objects. Comente los archivos de código fuente elaborados por usted, incluyendo *summary* con el propósito y *author* con su nombre y apellido. Comente los archivos de código fuente elaborados por usted aplicando JSDoc, incluyendo a nivel de clases e interfaces *summary* con el propósito y *author* con su nombre y apellido, así como documentación de métodos. Incluya en el archivo README.md, la información en inglés de la aplicación, descripción, features, dependencias y su información como autor. Empaquete su solución para el envío como un archivo **.zip** (único formato válido), con nombre *pc1**nrc**u**code**.zip* donde ***nrc*** es su número de NRC y ***code*** es su código de estudiante en minúsculas (por ejemplo *pc18130u20161C993*).

Nota: Cree su proyecto. No se permite refactoring de proyectos previos. Solo se considerará aptas para revisión y calificación soluciones con archivos creados integramente durante el horario de la evaluación. Solo se permite el uso del asistente del IDE de JetBrains para completar instrucciones durante la edición. No se permite AI Agentic Development de ningún tipo.

Fuera del alcance: Sidebar**,** Routing

Referencias:

https://fakestoreapi.com/
https://primevue.org/installation
https://primevue.org/button
https://primevue.org/toolbar
https://primevue.org/toast
https://primeflex.org/installation
https://primeflex.org/gridsystem
https://primeflex.org/flex
https://axios-http.com/docs/intro
https://vue-i18n.intlify.dev

Rúbrica de calificación

Criterio de Calificación	Sobresaliente (S)	Esperado (E)	Necesita Mejorar (M)	Insuficiente (I)	Calificación
C01. Building y ejecución	Al abrir el proyecto y ordenar la ejecución, ésta se inicia sin problemas. La aplicación es accesible en la ruta indicada.	La aplicación no llega a iniciar y ejecutarse, sin embargo el proceso de building llega a concluir.	Al cargar el proyecto el proceso de building presenta errores y no llega a concluir.	No elabora solución	
	2.0 puntos	**1.0 punto**	**0.5 puntos**	**0 puntos**	
C02. User Interface	Se evidencia que la interfaz de usuario aplica Responsive Web Design. La interfaz cumple con las características solicitadas para la estructura, elementos de la interfaz de usuario.	La interfaz de usuario no evidencia con claridad que aplica Responsive Web Design, o no cumple con todas las características solicitadas.	La aplicación no implementa una interfaz de usuario con las características solicitadas.	La aplicación no presenta la información indicada.	
	3.0 puntos	**2.0 puntos**	**1.0 puntos**	**0 puntos**	
C03. Features	La vista cumple de forma completa con los requisitos de interfaz de usuario, con todas las características y restricciones indicadas en el enunciado. Se cumple que la información es recuperada desde el endpoint solicitado.	La vista cumple de forma parcial con los requisitos de interfaz de usuario, o con la mayoría de las características y restricciones indicadas en el enunciado. Se cumple que la información es recuperada desde el endpoint solicitado.	La vista cumple de forma parcial con los requisitos de interfaz de usuario, con solo algunas de características y restricciones indicadas en el enunciado, o se cumple que la información es recuperada desde el endpoint solicitado, sin embargo, esta no se visualiza.	No se implementa el feature.	
	7.0 puntos	**4.5 puntos**	**2.0 puntos**	**0 puntos**	
C04. Code Organization	El desarrollador organiza el código y los elementos de frontend de la solución, aplicando buenas prácticas de JavaScript y Vue, agrupando los elementos de la solución según convenciones, manteniendo organización de paquetes y carpetas recomendadas por el fabricante y buenas prácticas de la industria de software, bajo un enfoque domain-driven, cumpliendo con las restricciones técnicas sobre organización.	El desarrollador aplica en frontend un enfoque domain-driven, la mayoría de convenciones, recomendaciones y buenas prácticas de JavaScript o Vue, o no cumple con todas las restricciones técnicas sobre organización.	El desarrollador aplica en frontend solo algunas de las convenciones, recomendaciones y buenas prácticas de JavaScript o Vue, o no cumple con las restricciones técnicas sobre organización.	No se evidencia un criterio de organización para los elementos de la solución, limitándose a la estructura por defecto.	
	3.0 punto	**2.0 puntos**	**1.0 puntos**	**0 puntos**	
C05. Code Quality	Utiliza para el frontend el lenguaje de programación JavaScript, el framework Vue y cumple con las restricciones técnicas indicadas. La codificación tiene un estilo claro, indentando los bloques de código según los estándares de programación correspondientes al lenguaje, aplicando una lógica consistente en los métodos, condicionales sin escenarios no contemplados, uso adecuado de reutilización de código para evitar redundancia. Aplica patrones de arquitectura y patrones de diseño. Distribuye el código en los niveles correspondientes, asignando lógica de persistencia, lógica de negocio, lógica de control, y transferencia a las interfaces y clases que corresponden.	Utiliza para el frontend el lenguaje de programación JavaScript y el framework Vue. La codificación es funcional, pero cumple de forma parcial con las restricciones técnicas, o sólo aplica parcialmente los estándares de indentación de bloques de código, ó existen ineficiencias en la codificación: redundancia ó inconsistencias en la lógica de programación. Aplica parcialmente patrones de arquitectura y patrones de diseño, o existe en algunas partes una distribución de la lógica en los niveles incorrectos.	Cumple con solo algunas restricciones técnicas, o no se evidencia aplicación de estándares ó criterios de eficiencia en la codificación, con ausencia de comentarios, ó no aplica patrones de arquitectura ni patrones de diseño, aunque la codificación es funcional.	No utiliza el lenguaje de programación JavaScript o no utiliza Vue, o no cumple con la mayoría de restricciones técnicas o el código no es funcional.	
	3.0 puntos	**2.0 punto**	**1.0 puntos**	**0 puntos**	
C06. Naming Standards	El desarrollador aplica en todos los nombres de objetos de programación como paquetes, componentes, interfaces, clases, objetos, variables, constantes y métodos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador aplica en la mayoría de casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador aplica sólo en algunos casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador no aplica nomenclatura en inglés para los objetos de programación ó recursos.	
	2.0 puntos	**1.0 punto**	**0.5 puntos**	**0 puntos**	
Total	**20 puntos**	**12.5 puntos**	**6.00 puntos**		

Lima, 23 de septiembre del 2026